

DSA Project

H Vidith

24b1095

B Charan

24b0966

T Sri Chandravadan

24b0931

Contents

1	Introduction	4
2	Directory Structure of Code	5
3	Makefile and Targets	10
3.1	Explanation of Targets	10
3.2	How to Use the Makefile	12
4	Assumptions	12
5	Test Cases and Analysis	13
5.1	Phase 1 Test Cases: Shortest Path, Dynamic Updates, and KNN . . .	13
5.2	Phase 2 Test Cases: Advanced Pathfinding	16
5.3	Test 1 — Disconnected Graph (Unreachable Orders)	18
5.4	Test 2 — Pickup equals Dropoff (Zero-distance order)	19
5.5	Test 3 — Single Driver, Many Orders (Stress / Performance)	20
5.6	Test 4 — Chain Delivery (Dropoff = next pickup)	20
5.7	Test 5 — Invalid Node IDs (Nonexistent nodes)	21
6	Python Scripts and Libraries	22
7	Time and Space Complexity	23
7.1	Phase [1] part 1: [Shortest Path]	23
7.2	Phase [1] part 2: [K Nearest Neighbors]	24
7.3	Phase [2] part 1: [K Shortest paths(Exact)]	24
7.4	Phase [2] part 2: [K Shortest paths(Heuristic)]	24
7.5	Phase [2] part 3: [Approximate Shortest paths]	25
8	Approach for Each Query Type	25
8.1	Phase [1] part 1: [Shortest Path] Query Type 1: [Minimizing Distance]	25
8.2	Phase [1] part 1: [Shortest Path] Query Type 2: [Minimizing Time]	26

8.3	Phase [1] part 1: [Shortest Path] Query Type 3: [Constraints]	27
8.4	Phase [1] part 2: [K Nearest Neighbors] Query Type 1: [Euclidean distance]	29
8.5	Phase [1] part 2: [K Nearest Neighbors] Query Type 2: [Shortest path distance]	29
8.6	Phase [2] part 1: [K Shortest paths] Query Type : [Exact]	30
8.7	Phase [2] part 2: [K Shortest paths] Query Type : [Heuristic]	32
8.8	Phase [2] part 3: [Approximate Shortest paths] Query Type : [Distance]	33
9	Phase 3 Code Implementation	35
9.1	Data Structures for Logistics (<code>graph.h</code>)	35
9.2	Shortest Path Algorithm (<code>algos.cpp</code>)	36
9.3	Greedy Round-Robin Scheduling (<code>graph.cpp</code>)	37
9.4	Time and Space Complexities of Implemented Functions	40
9.4.1	Greedy Delivery Scheduler (<code>Graph::delivery</code>)	40
9.4.2	Complexity Comparison (Theoretical vs. Actual Code)	42
10	AI Chat	42

1 Introduction

The project concentrates on the following concepts :

How do we solve the problem of navigation?

At what cost ?

Efficiency ?? (Cost Vs Accuracy)

How is this problem addressed presently ??

2 Directory Structure of Code

Provide a clear description of the directory layout of your project. Mention the purpose of each major folder and file so that the evaluator can easily understand where different components of your code reside.

Example Directory Structure

```
/
├── phase1
│   ├── inc
│   │   ├── edge.h
│   │   ├── graph.h
│   │   └── node.h
│   ├── read_json
│   │   ├── inc
│   │   │   └── read.h
│   │   ├── read_graph.cpp
│   │   └── read_query.cpp
│   └── src
│       ├── algos.cpp
│       ├── edge.cpp
│       ├── graph.cpp
│       ├── knn.cpp
│       ├── node.cpp
│       └── sample.cpp
├── phase2
│   ├── inc
│   │   ├── edge.h
│   │   ├── graph.h
│   │   └── node.h
│   ├── read_json
│   │   ├── inc
│   │   │   └── read.h
│   │   ├── read_graph.cpp
│   │   └── read_query.cpp
│   └── src
│       ├── algos.cpp
│       ├── edge.cpp
│       ├── graph.cpp
│       ├── node.cpp
│       └── sample.cpp
├── phase3
├── Makefile
├── nlohmann
│   └── json.hpp
```

```
phase3
├── inc
│   ├── edge.h
│   ├── graph.h
│   └── node.h
├── read_json
│   ├── inc
│   │   └── read.h
│   ├── read_graph.cpp
│   └── read_query.cpp
├── src
│   ├── algos.cpp
│   ├── edge.cpp
│   ├── graph.cpp
│   └── node.cpp
└── sample.cpp
```

Contains What?**PHASE1**

node.h: It has the Class declaration of the Node.

edge.h: It has the Class declaration of the Edge.

graph.h: It has the Class declaration of the graph.

read.h: It has the function declaration which are in `read_graph` and the `read_query`.

read_graph.cpp: which gives the graph class as output from the `graph.json`.

read_query: which gives the result after computations from the queries.

algos.cpp: Contains the Dijkstra and the Astar algorithms.

edge.cpp: This file contains definitions of the functions which are declared in the `edge.h`.

graph.cpp: This file contains definitions of the functions which are declared in the `graph.h`.

knn.cpp: This file contains functions which are part of the graph class that are used for the computing the K-nearest neighbours.

node.cpp: this file contains functions which are declared in the `node.h`.

sample.cpp : This file calls the `read_graph` and `read_query` function .

PHASE2

node.h: It has the Class declaration of the Node.

edge.h: It has the Class declaration of the Edge.

graph.h: It has the Class declaration of the graph.

read.h: It has the function declaration which are in read_graph and the read_query.

read_graph.cpp: which gives the graph class as output from the graph.json

read_query: which gives the result after computations from the queries

algos.cpp: Contains the Dijkstra and the Astar algorithms.

edge.cpp: This file contains definitions of the functions which are declared in the edge.h.

graph.cpp: This file contains definitions of the functions which are declared in the graph.h.

node.cpp: this file contains functions which are declared in the node.h.

sample.cpp: this file calls the read_graph and read_query function.

PHASE3

node.h: It has the Class declaration of the Node.

edge.h: It has the Class declaration of the Edge.

graph.h: It has the Class declaration of the graph.

read.h: It has the function declaration which are in read_graph and the read_query.

read_graph.cpp: which gives the graph class as output from the graph.json

read_query: which gives the result after computations from the queries

algos.cpp: Contains the Dijkstra and the Dijkstra_oneway algorithms.

edge.cpp: This file contains definitions of the functions which are declared in the `edge.h`.

graph.cpp: This file contains definitions of the functions which are declared in the `graph.h`.

node.cpp: this file contains functions which are declared in the `node.h`.

sample.cpp: this file calls the `read_graph` and `read_query` function.

Makefile: This has the code which builds the binary files (`phase1` and `phase2`) in the parent directory.

nlohmann: This has the `json.hpp` file which is provided to operate with json files.

3 Makefile and Targets

3.1 Explanation of Targets

The Makefile defines several targets to streamline the build process. Each target serves a specific purpose as described below.

1. **all**

This is the default target. It performs the following tasks automatically:

- Creates the `build/` directory if it does not exist.
- Compiles all source files for Phase 1, Phase 2, and Phase 3.
- Generates three executables: `phase1`, `phase2`, and `phase3`.

Usage:

```
make
```

2. **build directory**

Before compiling any objects, the Makefile ensures the existence of the directory:

- `build/` – all object files (`*.o`) are stored here.

This directory is created by:

```
make BUILD_DIR
```

but typically it is invoked automatically when you run `make all`.

3. **Phase Targets (`phase1`, `phase2`, `phase3`)**

Each phase has its own binary:

- `phase1`

- `phase2`
- `phase3`

Each target links the corresponding object files inside `build/` to produce the final executable.

Usage:

```
make phase1
make phase2
make phase3
```

4. Object File Compilation

For every C++ source file in each phase, a dedicated rule exists:

- Compiles with `g++`, `-O3` optimization, warnings enabled, and C++17 standard.
- Includes phase-specific include directories.

Example:

```
build/p1_graph.o: phase-1/src/graph.cpp
```

These rules are triggered automatically when building any phase.

5. `clean`

This target removes all generated files:

- Deletes the `build/` directory.
- Deletes the binaries: `phase1`, `phase2`, `phase3`.

Usage:

```
make clean
```

6. rebuild

This sequence fully resets and rebuilds the project. It performs:

1. `clean`
2. `all`

Usage:

```
make rebuild
```

This ensures a completely fresh compilation.

3.2 How to Use the Makefile

- To compile everything: `make`
- To compile only one phase: `make phase1` (or `phase2/phase3`)
- To delete all build files: `make clean`
- To fully rebuild: `make rebuild`

This modular structure ensures each phase can be built independently while also supporting a unified build process for the entire project.

4 Assumptions

- **Input Format:**
- **Data Constraints:**
- **Memory Availability:**
- **System Environment:**
- **Phase 3 Specific:**

5 Test Cases and Analysis

The provided test suite, based on `graph.json` and `queries.json`, is comprehensive, targeting critical edge cases for shortest path, dynamic updates, K-Nearest Neighbors (KNN), and advanced pathfinding algorithms. The following analysis is structured by functional phase.

5.1 Phase 1 Test Cases: Shortest Path, Dynamic Updates, and KNN

1.1 Shortest Path Edge Cases

These queries test the robustness of the core shortest path algorithm (e.g., Dijkstra's) under various constraints and scenarios. The base graph is weighted by edge `length` for distance queries and `average_time` for time queries.

|C0.6cm|L1.5cm|C1.8cm|C1cm|L4cm|L5.6cm| Shortest Path Edge Cases Analysis

ID	Type	Source	→ Target	Mode	Constraints	Verification Focus
----	------	--------	----------	------	-------------	--------------------

ID	Type	Source	→ Target	Mode	Constraints	Verification Focus
----	------	--------	----------	------	-------------	--------------------

1	shortest_path	0	→ 0	Distance	None	Source equals Target (Zero-length path).
---	---------------	---	-----	----------	------	---

4	shortest_path	0	→ 11	Distance	None	Unreachable Target (Node 11 is isolated).
---	---------------	---	------	----------	------	--

5	shortest_path	0	→ 8	Distance	forbidden_nodes: [4]	Forbidden Node (Forces path 0 → 1 → 2 → 5 → 8 or 0 → 3 → 4 → 7 → 8). Checks if the algorithm correctly skips a node.
---	---------------	---	-----	----------	----------------------	---

6	shortest_path	0	→ 8	Distance	forbidden_nodes: [4, 5, 7]	Disconnected/Heavily Constrained (Tests if the path still exists after removing several key grid nodes).
---	---------------	---	-----	----------	----------------------------	---

7 `shortest_path 0 → 8` Distance `forbidden_road_types: ["primary"]` **Forbidden Road Type** (Forces use of secondary/tertiary/local roads).

9 `shortest_path 0 → 8` Distance `forbidden_nodes: [4], forbidden_road_types: ["expressway"]` **Combined Constraints** (Ensures both node and road constraints are applied simultaneously).

10 `shortest_path 0 → 8` Distance `forbidden_nodes: [1..7, 9, 10]` **Extreme Constraining** (Should result in "No path" or test fallback to non-obvious routes).

11 `shortest_path 8 → 5` Distance `None` **One-way Traversal (Reverse)** (The path $5 \rightarrow 9 \rightarrow 8$ involves one-way edges. Query checks $8 \rightarrow 5$, which should not use $9 \rightarrow 8$ or $5 \rightarrow 9$).

12 `shortest_path 5 → 8` Distance `None` **One-way Traversal (Forward)** (Should use $5 \rightarrow 9$ and $9 \rightarrow 8$ for a likely short path).

1.2 Dynamic Updates Edge Cases

These queries verify the graph's ability to handle modifications (add, remove, change edge attributes) and that the shortest path algorithm adapts correctly.

|C0.6cm|L1.7cm|C1.5cm|L4.2cm|L6.9cm| Dynamic Updates Edge Cases Analysis

ID	Type	Edge ID	Patch/Action	Verification Focus
----	------	---------	--------------	--------------------

ID	Type	Edge ID	Patch/Action	Verification Focus
----	------	---------	--------------	--------------------

13 `remove_edge 1013` ($0 \leftrightarrow 4$) Remove Initial check for **edge removal** functionality.

14 `shortest_path N/A` Query $0 \rightarrow 4$ Checks if the path changes after edge 1013 is removed.

15 `remove_edge 1013` Remove **Double removal** (Should handle graceful failure)

or simply confirm removal).

16 `modify_edge 1013 {"length": 150.0}` **Modify a removed edge** (Implicitly restores it with new length).

17 `shortest_path N/A` Query $0 \rightarrow 4$ Checks new shortest path using the modified length of 1013.

19 `modify_edge 9999 {"length": 100.0}` **Non-existent Edge Modification** (Should fail gracefully).

21 `remove_edge 1001 ( $0 \leftrightarrow 1$ )` Remove Prepares for a sequential update test.

23 `shortest_path N/A` Query $0 \rightarrow 1$ Confirms the removal's effect on the shortest path.

24 `modify_edge 1001 {"length": 100.0, "average_time": 10.0}` **Restore/Modify** (Puts edge 1001 back with original values).

25 `shortest_path N/A` Query $0 \rightarrow 1$ Confirms path correctness after restoration.

47 `remove_edge 1016 ( $0 \leftrightarrow 10$ )` Remove Part of a sequential test to isolate node 10.

48 `remove_edge 1017 ( $10 \leftrightarrow 4$ )` Remove Completes isolation of node 10.

49 `shortest_path N/A` Query $0 \rightarrow 10$ Should return **"No path exists"**.

50 `modify_edge 1016 {"length": 70.71}` **Restore** edge 1016.

51 `shortest_path N/A` Query $0 \rightarrow 10$ Should return a path now that 1016 is restored.

1.3 K-Nearest Neighbors (KNN) Edge Cases

These queries test the KNN functionality across both Euclidean (straight-line) and Shortest Path metrics, including edge cases like limited POIs and off-node queries.

|C0.6cm|L1.5cm|C1.5cm|C1.5cm|C0.5cm|L1.5cm|L6.2cm| KNN Edge Cases Analysis

ID	Type	POI	Query Point	k	Metric	Verification	Focus
----	------	-----	-------------	-----	--------	--------------	-------

ID	Type	POI	Query Point	k	Metric	Verification	Focus
----	------	-----	-------------	-----	--------	--------------	-------

27 knn restaurant (0.0,0.0) 10 Euclidean **k larger than available POIs**
(Tests if the result is capped at the number of available POIs).

30 knn pharmacy (0.0,0.0) 1 Euclidean **Minimum Case** ($k = 1$) for Euclidean distance.

28 knn hospital (1.0,1.0) 2 Euclidean **Typical case** for Euclidean distance.

31 knn restaurant (0.0,0.0) 3 Shortest Path **Shortest Path Metric** starting from a POI node (Node 0).

32 knn hospital (0.5,0.5) 2 Shortest Path **Query Point Not on a Node**
(Requires routing the query point to the nearest graph node for SP calculation).

5.2 Phase 2 Test Cases: Advanced Pathfinding

2.1 K-Shortest Paths (Exact)

These queries test finding the k most optimal paths, crucial for applications needing multiple route options.

|C0.6cm|L1.7cm|C1.8cm|C0.5cm|C1cm|L8.4cm| K-Shortest Paths (Exact) Analysis

ID Type Source $\rightarrow$ Target k Mode Verification Focus

ID Type Source $\rightarrow$ Target k Mode Verification Focus

35 k_shortest_paths $0 \rightarrow 8$ 3 Distance **Find top 3 distinct paths** using an exact algorithm (e.g., Yen's).

37 k_shortest_paths $0 \rightarrow 1$ 2 Distance **Finding multiple paths in a simple structure** (Checks for correct handling of alternate routes: $0 - 1$ vs $0 - 4 - 1$ vs $0 - 3 - 4 - 1$).

38 k_shortest_paths $0 \rightarrow 4$ 4 Distance **Paths with Ties** (e.g., $0 \rightarrow 10 \rightarrow 4$

vs $0 \rightarrow 3 \rightarrow 4$). Ensures stable ranking for equal-length paths.

39 k_shortest_paths $0 \rightarrow 11$ 2 Distance **K-shortest to Unreachable Node**
(Node 11 is isolated; should return 0 paths).

2.2 K-Shortest Paths Heuristic (Distinct Paths)

These queries test finding diverse paths using a heuristic to manage overlap between solutions.

|C0.6cm|L2.5cm|C1.8cm|C0.5cm|C2.5cm|L6.2cm| K-Shortest Paths Heuristic Analysis

ID	Type	Source $\rightarrow$ Target	k	Overlap Threshold	Verification Focus
----	------	-----------------------------	-----	-------------------	--------------------

ID	Type	Source $\rightarrow$ Target	k	Overlap Threshold	Verification Focus
----	------	-----------------------------	-----	-------------------	--------------------

40	k_shortest_paths_heuristic	$0 \rightarrow 8$	3	50%	Typical Heuristic Case (Find 3 paths, allowing moderate path overlap).
----	----------------------------	-------------------	---	-----	--

41	k_shortest_paths_heuristic	$0 \rightarrow 8$	5	30%	Low Overlap (Stricter constraint to force highly diverse routes).
----	----------------------------	-------------------	---	-----	--

42	k_shortest_paths_heuristic	$0 \rightarrow 8$	2	70%	High Overlap (More lenient constraint, likely returning paths very similar to the true shortest path).
----	----------------------------	-------------------	---	-----	---

2.3 Approximate Shortest Path (ASP)

These queries test the trade-off between speed and accuracy by setting a time budget and an acceptable error margin.

|C0.6cm|L2.5cm|C3.5cm|C1.8cm|C2.5cm|L4.5cm| Approximate Shortest Path Analysis

ID	Type	Queries	Time Budget	Acceptable Error	Verification Focus
----	------	---------	-------------	------------------	--------------------

ID	Type	Queries	Time Budget	Acceptable Error	Verification Focus
----	------	---------	-------------	------------------	--------------------

43 approx_shortest_path $0 \rightarrow 8, 0 \rightarrow 2, 3 \rightarrow 9$ 100ms 10.0% **Batch Queries, typical budget** (Tests if the approximate solution finds paths within the error bounds for known reachable nodes).

44 approx_shortest_path $0 \rightarrow 11$ 50ms 5.0% **Unreachable Target in ASP** (Tests if the approximate algorithm handles isolated nodes and returns an appropriate error/result).

Overview

Below are five edge test cases chosen to break common fragile implementations for Phase 3 (routing / assignment). Each test includes the JSON input and a short explanation of why many student implementations fail on it.

5.3 Test 1 — Disconnected Graph (Unreachable Orders)

JSON

```
{
  "orders": [
    { "order_id": 1, "pickup": 0, "dropoff": 5 },
    { "order_id": 2, "pickup": 10, "dropoff": 15 }
  ],
  "fleet": { "num_delivery_guys": 2, "depot_node": 0 }
}
```

Why this breaks implementations

- Dijkstra/A* returns “no path” (commonly represented as -1.0 or INF) for orders in a different component.

- Naïve code often tries to use that distance as an index or to build a route and crashes (out-of-range lookups).
- Robust behavior: detect unreachable pickups/dropoffs, skip or report them, and never dereference invalid paths.

5.4 Test 2 — Pickup equals Dropoff (Zero-distance order)

JSON

```
{
  "orders": [
    { "order_id": 1, "pickup": 100, "dropoff": 100 },
    { "order_id": 2, "pickup": 200, "dropoff": 300 }
  ],
  "fleet": { "num_delivery_guys": 2, "depot_node": 0 }
}
```

Why this breaks implementations

- Zero-length subpaths (pickup==dropoff) can cause duplicate node entries in routes (e.g., [0,100,100,...]).
- Some code treats zero distance as “error” or filters it out and then loses the associated order.
- Time / cost calculations may divide by distance or rely on strictly positive distances, producing incorrect totals.
- Robust behavior: explicitly allow and handle zero-length legs, and ensure order accounting still records the completed delivery.

5.5 Test 3 — Single Driver, Many Orders (Stress / Performance)

JSON (example shortened – expand to 50 orders when running)

```
{
  "orders": [
    { "order_id": 1, "pickup": 100, "dropoff": 200 },
    { "order_id": 2, "pickup": 150, "dropoff": 250 },
    { "order_id": 3, "pickup": 120, "dropoff": 220 },
    ...
    { "order_id": 50, "pickup": 555, "dropoff": 666 }
  ],
  "fleet": { "num_delivery_guys": 1, "depot_node": 0 }
}
```

Why this breaks implementations

- Naïve implementations call shortest-path computations repeatedly without caching (e.g., $O(n^2)$ *Dijkstra* calls) and timeout.
- Memory and repeated allocations can also cause significant slowdowns.
- Robust behavior: cache pairwise distances, reuse computed paths, or use efficient heuristics and pruning.

5.6 Test 4 — Chain Delivery (Dropoff = next pickup)

JSON

```
{
  "orders": [
    { "order_id": 1, "pickup": 100, "dropoff": 200 },
    { "order_id": 2, "pickup": 200, "dropoff": 300 },

```

```
{ "order_id": 3, "pickup": 300, "dropoff": 400 }  
],  
"fleet": { "num_delievery_guys": 1, "depot_node": 0 }  
}
```

Why this breaks implementations

- Optimal route exploits chaining: delivering one order lands the driver exactly at the next pickup.
- Greedy pickup-first heuristics may miss chaining opportunities and produce longer routes.
- Robust behavior: algorithms should consider combined pickup/delivery sequences or at least detect chain patterns and avoid re-traversing common segments.

5.7 Test 5 — Invalid Node IDs (Nonexistent nodes)

JSON

```
{  
  "orders": [  
    { "order_id": 1, "pickup": 99999, "dropoff": 200 }  
  ],  
  "fleet": { "num_delievery_guys": 1, "depot_node": 0 }  
}
```

Why this breaks implementations

- The pickup node does not exist in the graph; many implementations assume nodes referenced by orders are valid.
- Attempting to run shortest-path or index into node arrays causes out-of-range errors or crashes.

- Robust behavior: validate all input node IDs against the graph before any computation and reject or report invalid orders.

Notes for testers:

- For performance tests (Test 3), run with realistic graph sizes and ensure caching is used; measure end-to-end runtime.
- For unreachable/invalid tests (Tests 1,5), decide expected behavior in your spec (skip with warning, or return a particular error field) and assert that implementations follow it.
- Use both distance and time modes if your code supports both; edge cases can behave differently in time-optimized runs (e.g., zero-length edges vs. time-profile edges).

6 Python Scripts and Libraries

The testcase generator given is written in python language, which in specific send a request to the open street maps and gets the data of the specified grid of the real world map. This grid is given is specified in side the generator file.

running instructions:

For running or generating the testcases we modify the Lata, Latb and Lona, Lonb required in the request sent to the open street map (inside the python file), and run the testcase generator python file. The results or ouputs will be a "graph.json" in the same folder as the testcase generator file and the queries will be for 3 phases separately in a folder "queries" .

libraries included:

requests: This library is used to send the request to the open street website.

json: This library is used to read json data files.

os: This library is used for the path concatenation and to create files and directories

random: This library is used to generate queries from the given graph and also used for some probabilities say each type should occur in the query with this probability to get all types of queries. And for the speed profiles and avg speeds.

NOTE1: as this testcase generator completely depends on the real world maps and that is given by open street there will be cases when there may be no edges in the given region then it will output the graph but no queries, it will print that no edges to the terminal.

NOTE2: And there will be case when the website may not respond correctly so we have to wait and run the testcase generator again.

7 Time and Space Complexity

7.1 Phase [1] part 1: [Shortest Path]

- **Time Complexity:**

- **Construction/Preprocessing:** Nothing
- **Query Type Minimizing Distance:** $O((E) * \log V)$: Astar function using edge lengths as weights. (Heuristic applied)
- **Query Type Minimizing Time:** $O((E) * \log V)$: Astar function using time as weights. (Heuristic not applied)
- **Query Type Constraints:** Applying constraints on edges and nodes is constant ($O(1)$) time done in sample.cpp.

- **Space Complexity:**

- **Data Structures:** $O(E)$ for Min-heap.
- **Auxiliary Space:** $O(V)$ for Vectors used.

7.2 Phase [1] part 2: [K Nearest Neighbors]

- **Time Complexity:**
 - **Construction/Preprocessing:** Nothing.
 - **Query Type Euclidean distance:** $O(k * V)$: Traversing the Nodes.
 - **Query Type Shortest path distance:** $O(k * (E) * \log V)$: Applying Dijkstra k times.
- **Space Complexity:**
 - **Data Structures:** $O(E)$ for Min-heap.
 - **Auxiliary Space:** $O(V)$ for Vectors used.

7.3 Phase [2] part 1: [K Shortest paths(Exact)]

- **Time Complexity:**
 - **Construction/Preprocessing:** Nothing.
 - **Query Type k shortest paths:** $O(k * V * (E) * \log V)$: Removing each edge of previous short paths to find next short paths (Applying Astar)
- **Space Complexity:**
 - **Data Structures:** $O(k * V^2)$ for Max-heap.
 - **Auxiliary Space:** $O(k * V)$ for Vectors used.

7.4 Phase [2] part 2: [K Shortest paths(Heuristic)]

- **Time Complexity:**
 - **Construction/Preprocessing:** Nothing.
 - **Query Type heuristic:** $O(50 * k * E * \log V)$: Applying Astar 50 times (remaining code has less or equal time than this)

- **Space Complexity:**

- **Data Structures:** $O(M * E)$ for answer stored.
- **Auxiliary Space:** $O(K^2 * E)$ for Vectors used.

7.5 Phase [2] part 3: [Approximate Shortest paths]

- **Time Complexity:**

- **Construction/Preprocessing:** $O(C * (E) * \log V)$ Running Dijkstra C times.
- **Query Type distance:** $O(C * \sqrt{V} + \sqrt{V} * \log V)$: Applying Landmark Guided bidirectional search.

- **Space Complexity:**

- **Data Structures:** $O(E)$ for Min-heap.
- **Auxiliary Space:** $O(C * V)$ for Vectors used.

8 Approach for Each Query Type

8.1 Phase [1] part 1: [Shortest Path] Query Type 1: [Minimizing Distance]

Standard Algorithm:

We implemented the A* (A-Star) algorithm to efficiently compute optimal paths in weighted graphs. A* acts as an informed search strategy that improves upon standard Dijkstra's algorithm by incorporating a heuristic function. This function estimates the remaining cost from the current node to the target, allowing the search to prioritize promising paths and reducing the overall search space while guaranteeing optimality.

Implementation Details:

The frontier of nodes to be explored is managed using a `std::priority_queue` configured as a min-heap (using `std::greater`). We maintain two primary arrays: `sp` to store the actual cost from the source ($g(n)$) and `parent` to reconstruct the path.

Hueristic($h(n)$): We utilize Euclidean distance based on latitude and longitude coordinates, scaled as per Haver-Sign Formula to estimate the straight-line distance to the target.

Cost function($f(n)$): For Distance queries: The priority queue orders nodes based on $f(n) = g(n) + h(n)$, effectively applying the A* logic.

For Time queries: The heuristic is omitted, and nodes are ordered solely by accumulated time ($g(n)$). This effectively reduces the logic to Dijkstra's algorithm, ensuring correctness when dealing with dynamic edge weights (traffic profiles).

Path Reconstruction: Upon reaching the target, we backtrack from the destination to the source using the `parent` array to generate the final route.

8.2 Phase [1] part 1: [Shortest Path] Query Type 2: [Minimizing Time]

Standard Algorithm:

Same as Minimizing distance.(Hueristics not included for time)

Implementation Details:

Same as Minimizing distance.(Hueristics not included for time)

8.3 Phase [1] part 1: [Shortest Path] Query Type 3: [Constraints]

Standard Algorithm:

We initialised some functions in graph to modify edges and nodes as per the given possibilities of queries.(like `forbidden_roadtypes`, `remove_edge`, `forbidden_nodes`, `restore_nodes`, `restore_roadtypes`, `modify_edge`, `modify_road_type`, `modify_speedprofile`)

Implementation Details:

Here is the implementation of functions used in this query.

forbidden_roadtypes: This function efficiently handles batch constraints by utilizing pre-categorized vectors (primary, secondary, tertiary, etc.) populated during graph initialization. Instead of iterating through the entire edge list ($O(E)$), the function matches the input string to the corresponding category vector and iterates only through the relevant edges. It sets the boolean `is_valid` flag to false, effectively disabling these roads for any subsequent pathfinding queries without removing them from memory.

remove_edge: This implements a "soft delete" mechanism. The function first verifies the existence of the edge using the hash map's `.find()` method ($O(1)$ average time). If the edge exists and is currently valid, it toggles the state to invalid. This preserves the edge's ID and connectivity data for potential future restoration while preventing algorithms from traversing it.

forbidden_nodes: This function accepts a list of node IDs and directly accesses the Nodes data structure. It sets the validity flag of the specified nodes to false. This state change is checked by traversal algorithms (like Dijkstra or A*) before adding neighbors to the priority queue, effectively treating the node as an obstacle.

restore_nodes: Acting as the inverse of `forbidden_nodes`, this function iterates through the provided list of IDs and resets their `is_valid` status to true. This is typically called after a query is processed to clean up constraints and return the graph to its original state for the next query.

restore_roadtypes: This function reverses the operation of `forbidden_roadtypes`. It maps the input string identifiers back to the pre-categorized edge vectors. It iterates through the specific subset of edges associated with those types and resets their validity flag to true, re-enabling large sections of the graph efficiently.

modify_edge: This function handles dynamic weight updates via a flexible "patch" mechanism. It accepts a list of key-value pairs (attribute name and new value). It parses the string keys (e.g., "length", "average_time") to route the update to the correct member variable of the Edge object. Crucially, after applying the updates, it explicitly sets the edge's validity to true, allowing this function to simultaneously update and "re-open" a previously closed or removed road.

modify_road_type: This function updates the metadata classification of a specific edge. Changing this attribute ensures that subsequent calls to `forbidden_roadtypes` or logic depending on road hierarchy correctly include or exclude this edge based on its new classification.

modify_speedprofile: This function replaces the existing time-dependent speed vector for a specific edge with a new 96-slot profile. This update directly impacts time-based queries where traversal speed is a function of the arrival time at the node.

8.4 Phase [1] part 2: [K Nearest Neighbors] Query Type 1: [Euclidean distance]

Standard Algorithm:

For the K-Nearest Neighbors (KNN) query based on geometric proximity, we employ a linear scan over the relevant Points of Interest (POIs) combined with a **Max-Heap** of size k . This approach avoids sorting all distances ($O(N \log N)$) and instead maintains the k smallest distances found so far, resulting in a more efficient time complexity of $O(N \log k)$, where N is the number of POIs of the specific type.

Implementation Details:

The Euclidean distance is computed using the standard formula $\sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$ on the latitude and longitude coordinates. To find the nearest neighbors, we iterate through the vector of node IDs corresponding to the requested POI type (e.g., "Restaurant"). We push each node and its calculated distance into a `std::priority_queue`. Since the default C++ priority queue is a max-heap, if the queue size exceeds k , we `pop` the top element. This effectively discards the farthest node among the current candidates, ensuring the heap only retains the k closest nodes. Finally, we extract the elements and reverse the vector to return the neighbors in ascending order of distance.

8.5 Phase [1] part 2: [K Nearest Neighbors] Query Type 2: [Shortest path distance]

Standard Algorithm:

To find the K-Nearest Neighbors based on actual road network distance (shortest path) rather than simple geometric distance, we employ a two-step approach. First, we map the arbitrary input coordinate to the nearest graph vertex using a **Linear Nearest Neighbor Search**. Second, we execute **Dijkstra's Algorithm** starting from this vertex. Since Dijkstra's algorithm explores vertices in non-decreasing order

of distance, the first k nodes encountered that match the specific Point of Interest (POI) type are guaranteed to be the closest.

Implementation Details:

- **Coordinate Mapping (`get_nearestnode`):** The system iterates through all nodes in the graph ($O(V)$), calculating the Euclidean distance between the input latitude/longitude and each node's position. The node minimizing this distance is returned as the starting point (`u_id`) for the graph traversal.
- **Network Search (KNN):** We initialize a `std::priority_queue` as a min-heap with the start node.
 - As nodes are extracted from the priority queue (which guarantees they are visited in strict order of travel distance), we check the node's metadata for the requested POI string (e.g., "Petrol Station").
 - If a match is found, the node ID is added to the result list.
 - The algorithm utilizes an early exit condition: the loop terminates immediately once k POIs have been collected. This ensures we do not traverse the entire graph unnecessarily once the required neighbors are found.
 - Standard relaxation logic ($dist[v] > dist[u] + w$) is used to explore adjacent nodes.

8.6 Phase [2] part 1: [K Shortest paths] Query Type : [Exact]

Standard Algorithm:

To compute the Exact K-Shortest Paths, we implemented ****Yen's Algorithm****, a deviation-based method that systematically explores variations of the shortest path. It works by taking the $(k - 1)^{th}$ shortest path and deviating from it at each node (the "spur node") to find alternative routes. This algorithm relies on a standard shortest path algorithm (like Dijkstra or A*) as a subroutine to compute the spur paths.

Implementation Details:

We maintain a list of the k found shortest paths and a `std::priority_queue` (`potential_shortest`) to store candidate paths generated during deviations.

- **Outer Loop:** Iterates from 1 to K to find each successive shortest path.
- **Spur Node Iteration:** For the k^{th} path, we iterate through every node j in the $(k - 1)^{th}$ path, designating it as the *spur node*. The path from the source to the spur node is defined as the *root path*.
- **Graph Constraints (Modifications):**
 - To guarantee loopless paths, we temporarily set the validity of all nodes in the current *root path* (except the spur node) to **false**.
 - To ensure a unique path is found, we iterate through all previously discovered k paths. If a path shares the same root path, the edge leaving the spur node in that path is identified and temporarily set to **invalid**.
- **Pathfinding:** We execute our standard shortest path algorithm (Dijkstra or A*) from the spur node to the target. If a path exists, it is concatenated with the root path and pushed into the candidate priority queue.
- **State Restoration:** After processing each spur node, it is critical to restore the graph. We iterate through the lists of disabled nodes and edges, resetting their status to **valid** to ensure the graph is correct for the next iteration.
- **Selection:** Finally, the path with the minimum cost is extracted from the priority queue and added to our solution set.

8.7 Phase [2] part 2: [K Shortest paths] Query Type : [Heuristic]

Standard Algorithm:

For the Heuristic K-Shortest Paths query, where the goal is to find diverse but competitive routes, we implemented the **Iterative Penalty Method (IPM)** combined with a **Greedy Selection Strategy**. Unlike Yen's algorithm which finds strictly ordered paths by length (often resulting in negligible differences), IPM forces the search to explore different topologies by dynamically penalizing the edges of discovered paths. We generate a large pool of candidates and then select the optimal subset that minimizes a custom cost function balancing total distance and path overlap.

Implementation Details:

The implementation is divided into two distinct phases: Candidate Generation and Optimal Selection.

- **subpart 1: Candidate Pool Generation:** We run a loop (e.g., 50 iterations) to populate a `potential_shortest` pool.
 - In each iteration, we execute **A*** to find the current shortest path.
 - If the path is unique, it is added to the pool.
 - **Dynamic Penalization:** To force the discovery of alternative routes, we increase the weight of every edge in the current path. The penalty factor is dynamic, calculated as $1 + \frac{100}{\alpha \times (L-1)}$, where L is the path length. This ensures that edges are discouraged for future searches without completely disconnecting the graph.
 - We maintain a copy of the original graph weights to restore the state after the query.

- **subpart 2: Greedy Selection with Local Search:** From the pool of candidates, we select K paths that minimize the `Total_penalty`.
 - We always start with the true shortest path.
 - For the remaining $K - 1$ slots, we iteratively scan the candidate pool and greedily pick the path that minimizes the system's total penalty (a function of length deviation and edge overlap).
 - **Refinement (Swap Step):** To avoid getting trapped in a local minimum, after adding a path, we perform a "look-back" check. We test if replacing the *previously* selected path with a different candidate yields a lower total penalty for the set. If a better combination is found, we swap the paths.
- **Penalty Function (`Total_penalty`):** This helper function evaluates the quality of a selected set of paths. It computes a weighted sum of the **Distance Penalty** (how much longer a path is compared to the optimal) and the **Overlap Penalty** (counting pairs of paths that share significant segments).

8.8 Phase [2] part 3: [Approximate Shortest paths] Query Type : [Distance]

Standard Algorithm:

For the Approximate Shortest Path query, we implemented a **Center-based A* Search**, a highly efficient technique for accelerating point-to-point pathfinding. This method utilizes a small set of "center" nodes and precomputed distances to generate strong heuristic lower bounds for A* search using the Triangle Inequality. To further enhance performance, we employed **Bidirectional A* Search**, running simultaneous searches from the source and target, and utilized an approximation factor (ϵ) to terminate the search early once a path within the acceptable error margin is found.

Implementation Details:

- **Center Selection (centers):** We utilize a ****Farthest Point Sampling**** strategy to select C centers.
 - The first center is chosen arbitrarily (node 0).
 - Subsequent centers are selected iteratively by finding the node that maximizes the minimum distance to the existing set of centers. This ensures the centers are well-distributed across the graph, maximizing the effectiveness of the heuristic.
 - We use a helper function `Dijkstra_oneway` to compute these distances during the selection process.
- **Heuristic Computation (heuristic):** Based on the Triangle Inequality, for any center c , the distance between two nodes u and v is lower-bounded by $|dist(c, u) - dist(c, v)|$. We precompute a table of distances from all centers to all nodes. During the query, the heuristic $h(u, v)$ is calculated as the maximum value over all centers:

$$h(u, v) = \max_{c \in Centers} |dist(c, u) - dist(c, v)|$$

- **Bidirectional Approximate Search (approx_shortest_path):**
 - We maintain two priority queues: `fwd_queue` (Source $\rightarrow$ Target) and `bwd_queue` (Target $\rightarrow$ Source).
 - A variable `best_path_len` tracks the shortest path found so far where the two search frontiers meet.
 - **Early Termination:** Instead of running until the priority queues are empty or meet optimally, we calculate a dynamic `lower_bound` based on the current heuristic values in the priority queues. The search terminates immediately if the current best path satisfies the approximation

condition:

$$\text{best_path_len} \leq (1 + \epsilon) \times \text{lower_bound}$$

- This guarantees the returned path is within $\epsilon\%$ of the optimal length while significantly reducing computation time.

9 Phase 3 Code Implementation

This section documents the specific implementation of the **Greedy Round-Robin Scheduler** within the `Graph` class, as described in the theoretical explanation. The implementation is found in `graph.h`, `graph.cpp`, and `algos.cpp`.

9.1 Data Structures for Logistics (`graph.h`)

To model the Vehicle Routing Problem with Pickup and Delivery (VRPPD), custom structures were defined to manage orders and facilitate shortest path lookups.

Order Structure

The `Order` structure encapsulates the state of each delivery task.

```

1 struct Order{
2     long long id;
3     long long pick_up;
4     long long drop_off;
5     bool assigned; // True if a driver is currently assigned (not
6     strictly used in current greedy logic)
7     bool picked;   // True if the driver has picked up the order
8     bool recieved; // True if the order has been delivered
9     Order(long long id1, long long p1, long long d1):id(id1),pick_up(
10    p1),drop_off(d1),assigned(false),picked(false),recieved(false){}
11 };

```

Listing 1: Order Structure (`graph.h`)

Graph Class Declarations

The `Graph` class is extended to include the delivery function and standard Dijkstra functionality, which uses distance (`length` parameter set to `true` in `Dijkstra`).

```

1 class Graph{
2     // ... private members for graph structure ...
3     public:
4         // Dijkstra implementation for shortest path
5         std::pair<std::vector<long long>, long double> Dijkstra(
6             long long u_id, long long v_id, bool length);
7
8         // Main delivery scheduling function
9         std::vector<std::pair<std::vector<long long>, long double>>
10            delivery(long long depot, std::vector<Order>& orders, long long
11                num_drivers);
12 };

```

Listing 2: Relevant Graph Declarations (`graph.h`)

9.2 Shortest Path Algorithm (`algos.cpp`)

The foundation for all routing decisions is Dijkstra’s algorithm, which computes the shortest path between two nodes based on road length when the `length` flag is true.

```

1 std::pair<std::vector<long long>, long double> Graph::Dijkstra(long
2     long u_id, long long v_id, bool length){
3     // ... Initialization of priority queue, distance (sp), and
4     parent arrays ...
5
6     while(!unkown.empty()){
7         // ... Standard Dijkstra relaxation loop ...
8         for(auto edgeid: Adjacency[cur_node]){
9             // ... check for valid edge and next node ...
10            if(!visited[next_node]){
11                long double weight;
12                if(length){

```

```

11         weight=Edges[edgeid].get_length(); // Using
length as weight
12     }else{
13         // weight=Edges[edgeid].time_taken(sp[cur_node
]); // Time calculation (commented out/not fully implemented for
time in current code)
14     }
15     if(sp[next_node]>sp[cur_node]+weight){
16         parent[next_node]=cur_node;
17         sp[next_node]=sp[cur_node]+weight;
18         unkown.push({sp[next_node],next_node});
19     }
20 }
21 }
22 }
23 // ... Path reconstruction and return ...
24 }

```

Listing 3: Dijkstra Implementation Snippet (algorithms.cpp)

9.3 Greedy Round-Robin Scheduling (graph.cpp)

The `delivery` function implements the Greedy Round-Robin heuristic. Note that the provided code **does not include the caching/memoization mechanism** described in the theoretical explanation; it calls Dijkstra repeatedly for every distance calculation.

Driver Structure

A local `Driver` structure is used to maintain driver state, including current location and the set of picked-up orders.

```

1 struct Driver {
2     long long d_id;
3     std::vector<long long> route;
4     long long curr_loc;

```

```

5     std::set<long long> picked; // Set of order IDs currently held
    by the driver
6     long double total_time;
7     Driver(long long id, long long depot_loc): d_id(id), curr_loc(
    depot_loc), total_time(0.0) {route.push_back(depot_loc);}
8 };

```

Listing 4: Driver Structure within delivery (graph.cpp)

Assignment and Routing Logic

The core loop implements the Round-Robin assignment, where each driver greedily chooses the nearest task (dropoff or pickup).

```

1 std::vector<std::pair<std::vector<long long>, long double>> Graph::
    delivery(long long depot, std::vector<Order>& orders, long long
    num_drivers) {
2     // ... Driver initialization ...
3     long long delivered_count=0;
4
5     while (delivered_count<total_orders) {
6         bool ass=false;
7         for (long long d = 0; d < num_drivers; d++) {
8             long long best_order=-1;
9             long double best_dist=max_double;
10            bool is_pickup=false;
11
12            // Priority 1: Check Dropoffs for carried orders
13            for (long long picked_idx : drivers[d].picked) {
14                if (!orders[picked_idx].recieved) {
15                    auto result=Dijkstra(drivers[d].curr_loc, orders
    [picked_idx].drop_off, true); // No Caching
16                    long double distance=result.second;
17                    if (distance!=-1.0 && distance<best_dist) {
18                        best_dist=distance;
19                        best_order=picked_idx;
20                        is_pickup=false;

```

```

21         }
22     }
23 }
24
25     // Priority 2: Check Pickups if no dropoff found or
dropoff is too far
26     if (best_order==-1) {
27         for (long long o=0; o<orders.size();o++) {
28             if (!orders[o].picked && !orders[o].assigned) {
29                 auto result=Dijkstra(drivers[d].curr_loc,
orders[o].pick_up, true); // No Caching
30                 long double distance=result.second;
31                 if (distance!=-1.0 && distance<best_dist) {
32                     best_dist=distance;
33                     best_order=o;
34                     is_pickup=true;
35                 }
36             }
37         }
38     }
39
40     // Execute the best action (Pickup or Dropoff)
41     if (best_order!=-1 && best_dist!=max_double) {
42         ass=true;
43         if (is_pickup) {
44             // Update state for Pickup
45             drivers[d].route.push_back(orders[best_order].
pick_up);
46             drivers[d].curr_loc = orders[best_order].
pick_up;
47             drivers[d].total_time += best_dist;
48             orders[best_order].picked = true;
49             orders[best_order].assigned = true;
50             drivers[d].picked.insert(best_order);
51         } else {

```

```
52         // Update state for Dropoff
53         drivers[d].route.push_back(orders[best_order].
drop_off);
54         drivers[d].curr_loc = orders[best_order].
drop_off;
55         drivers[d].total_time += best_dist;
56         orders[best_order].recieved = true;
57         drivers[d].picked.erase(best_order);
58         delivered_count++;
59     }
60 }
61 }
62
63     if (!ass) break;
64 }
65
66     // ... Format and return results ...
67 }
```

Listing 5: Greedy Round-Robin Logic (graph.cpp)

9.4 Time and Space Complexities of Implemented Functions

9.4.1 Greedy Delivery Scheduler (Graph::delivery)

This function implements the Greedy Round-Robin heuristic, repeatedly calling `Dijkstra` to determine the next closest task (pickup or dropoff) for each driver.

Note: The current implementation in `graph.cpp` does not include caching.

Time Complexity (Without Caching)

The time complexity is dominated by the repeated calls to Dijkstra's algorithm within the main simulation loop:

$$T(\text{delivery}) = O(I \cdot K \cdot O \cdot (E \log N))$$

Where I is the number of total steps (iterations) required to complete all O orders (the total number of pickups and dropoffs is $2O$).

- **Outer Loop/Steps (I):** The total number of actions (moves) a driver makes is $2O$ (one pickup + one dropoff per order). Thus, $I \approx 2O$.
- **Inner Loop (Driver Iteration):** The algorithm iterates through all K drivers.
- **Greedy Search:** For each driver, it potentially scans O carried orders (for dropoffs) and O unpicked orders (for pickups). The scan size is $O(O)$.
- **Pathfinding:** For each potential task scanned, a full **Dijkstra** search is executed, costing $O(E \log N)$.
- **Simplified Upper Bound:** $T(\text{delivery}) \approx O(O^2 \cdot K \cdot E \log N)$.

Space Complexity

The space is required for the **Driver** structures, storing the final routes, and the local space used by repeated **Dijkstra** calls:

$$S(\text{delivery}) = O(N + O \cdot K)$$

- **Driver Storage:** $O(K)$ space for the driver states (current location, **picked** set).
- **Route Storage:** The final routes **route** can store up to $O(N)$ total nodes across all drivers (in the worst case where paths are long and distinct), or

$O(O \cdot N)$ if the full paths were stored, but since only the assignment nodes (pickup/dropoff) are stored in the route vector, the path length is bounded by the total number of assigned steps, $O(2O) \cdot (\text{Avg Path Length})$.

- ****Order Storage:**** $O(O)$ for the `orders` vector.

—

9.4.2 Complexity Comparison (Theoretical vs. Actual Code)

The theoretical explanation proposed using a distance cache to reduce the complexity of the delivery scheduler. If caching were fully implemented and effective, the complexity would shift:

- ****Dijkstra Calls with Caching:**** The number of `Dijkstra` calls is limited to the number of **unique** path pairs requested, $O(P^2)$ where P is the number of unique pickup/dropoff locations. The total complexity would be approximately:

$$T(\text{delivery}_{\text{cached}}) \approx O(P^2 \cdot E \log N + I \cdot K \cdot O)$$

(The first term is cache warming; the second is the $O(1)$ lookups).

10 AI Chat

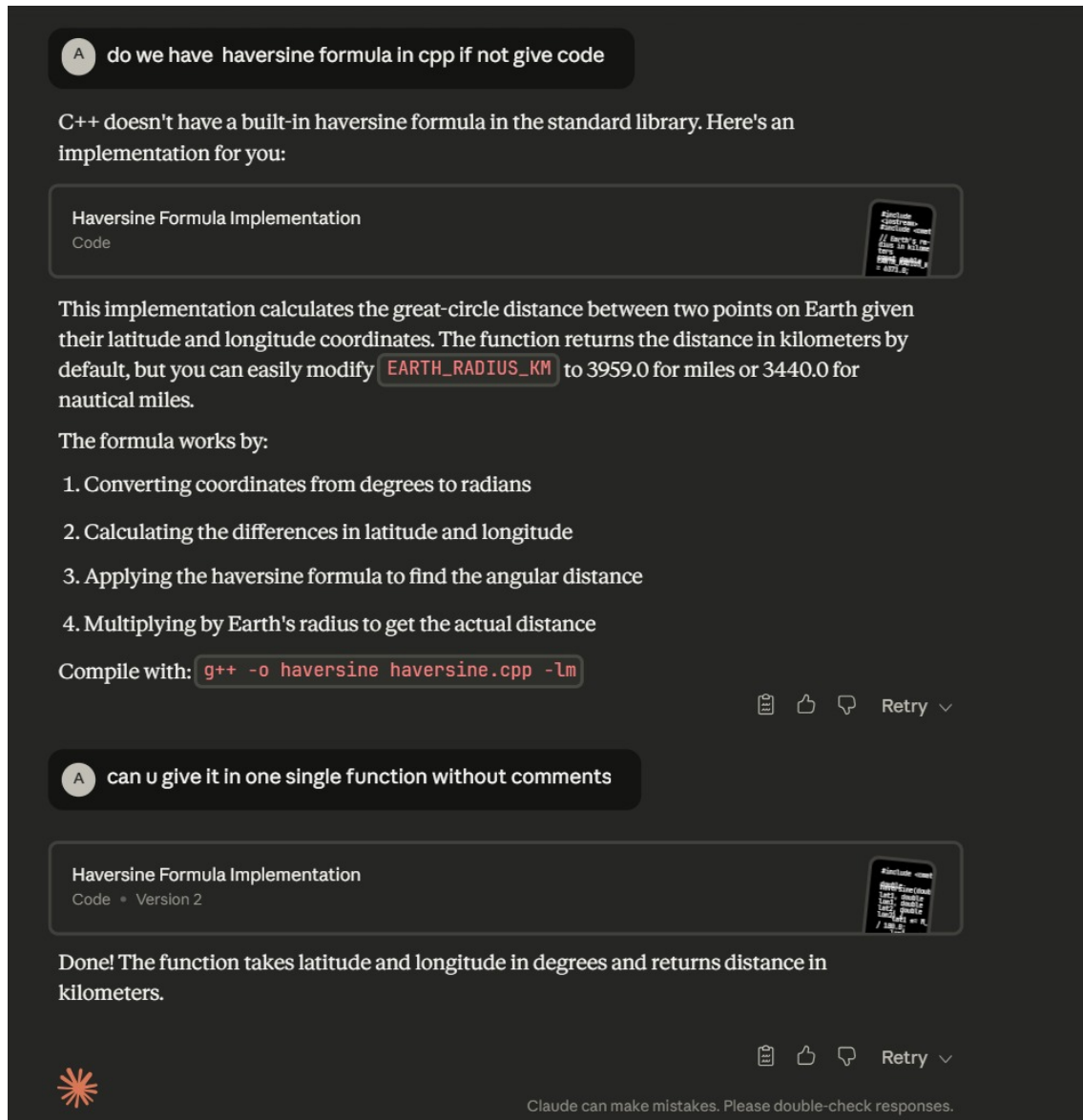


Figure 1:

lec-17-sp.pdf
545 lines
PDF

V

Abhi Jain 5 minutes ago
Spoiler: Read more about A star, you can play it safe there. There are conditions under which A-star will be correct and always produce correct results. (We have read A-star for 4 weeks, sadly.) Though we won't create testcases explicitly testing i

This appears to be a message from someone (Abhi Jain) providing a hint or clarification about the A* algorithm, likely in the context of a course assignment or exam.

Based on the lecture content and this message, here are the key points about A*:

When A* is Guaranteed to be Correct

The lecture explains two important conditions:

- 1. Admissibility (Definition 17.7):** The heuristic function $h(v)$ must be less than or equal to the actual shortest path distance from v to the target
 - Theorem 17.2:** If h is admissible, A* will find the optimal path
- 2. Consistency (Definition 17.8):** For each edge (v, k, w) , we need $h(v) \leq h(w) + k$
 - Theorem 17.3:** If h is consistent, each vertex will enter the openSet only once (making it behave more like Dijkstra's algorithm)

The "Play It Safe" Approach

The message suggests you can "play it safe" with A*. The safest approach mentioned in the lecture is:

- $h(v) = 0$ for all vertices $\rightarrow$ This reduces A* to Dijkstra's algorithm (Exercise 17.5a)
- This is always admissible and consistent
- It loses the goal-directed benefit but guarantees correctness

For Your Assignment/Exam

Since Abhi mentions they won't explicitly test A* (it's marked as "Not part of this course!" in the lecture), you likely don't need to implement a sophisticated heuristic. If you do use A*, using $h(v) = 0$ (essentially Dijkstra's) would be the safest choice.

Would you like help understanding any specific aspect of A* or Dijkstra's algorithm from the lecture?

Figure 2: